

Proyecto 1 - Redes

«Pequeña descripción de los objetivos del proyecto.»

Miembros del grupo

Carné	Nombre completo
2023064329	Luis Fernando Ureña Corrale
2023097390	Fabrizio Herrera Rodríguez
2024154899	Djedrielle Alexander Vargas
2023223291	Nicole Tatiana Parra Valverde

Módulos

- **DNS Interceptor:** Aplicación desarrollada en Rust que escucha en el puerto UDP/53. Esta aplicación recibe paquetes del protocolo DNS, los examina y siguiendo la especificación oficial del RFC2929, los procesa.
- **DNS API:**
- **DNS UI:** Interfaz web para crear, editar y eliminar registros DNS. También permite configurar health checks y administrar rangos IP por país.
- **Health Checker:**

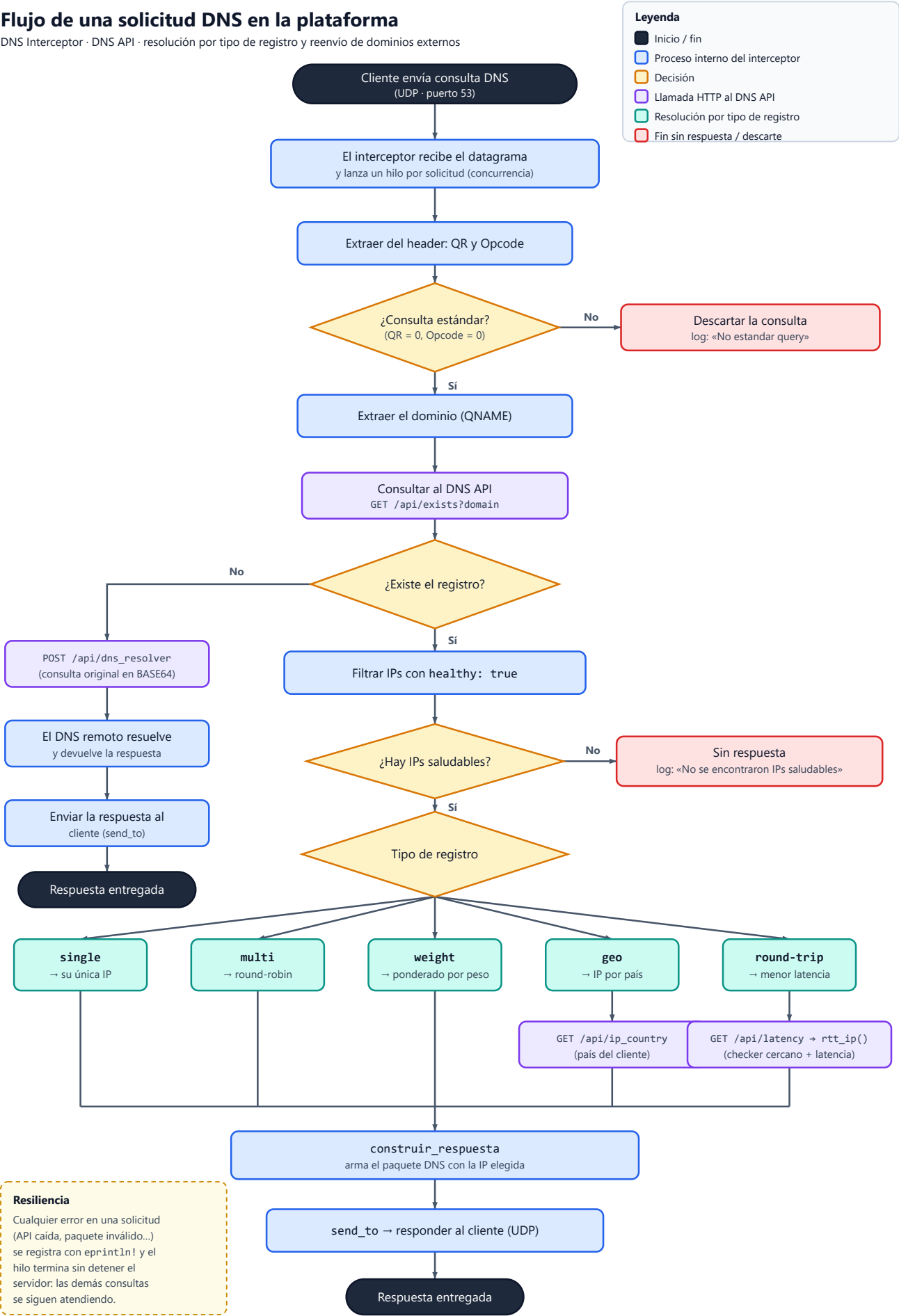
Ejecutar el proyecto

«Instrucciones de ejecución.»

Diagrama de Flujo

Flujo de una solicitud DNS en la plataforma

DNS Interceptor · DNS API · resolución por tipo de registro y reenvío de dominios externos



Estado de funcionalidades

Módulo	Funcionalidad	Estado	Observaciones
DNS Interceptor	Tipo de registro single	100%	Devuelve la única IP del registro.
DNS Interceptor	Tipo de registro multi	100%	Round-robin entre las IPs del registro.
DNS Interceptor	Tipo de registro round-trip	100%	IP de menor latencia según el checker más cercano; validado con datos simulados de un checker.
DNS Interceptor	Tipo de registro weight	100%	Distribución ponderada según el peso de cada IP.
DNS Interceptor	Tipo de registro geo	100%	Resuelve por país del cliente; en Docker el NAT da origen ZZ y usa la IP de respaldo (con IP pública real resuelve correcto).
DNS API	Funcionalidad		«Por completar por el responsable del módulo.»
Health Checker	Funcionalidad		«Por completar por el responsable del módulo.»
DNS UI	Registros DNS	100%	Permite crear, editar y eliminar los cinco tipos de registro.
DNS UI	Health checks	100%	Permite configurar pruebas TCP y HTTP para las IP de un registro.
DNS UI	Rangos IP por país	100%	Permite crear, editar y eliminar rangos de IP.

Pruebas realizadas

DNS Interceptor

Las pruebas se hicieron de forma incremental durante el desarrollo, desde la recepción de paquetes crudos hasta el uso del interceptor como DNS del sistema.

Requisitos previos para replicar:

- El **DNS Interceptor** corriendo en un contenedor, con el puerto UDP 53 del contenedor mapeado a un puerto del host (en desarrollo se usó **15353** porque el **53** y el **5353** estaban ocupados por **systemd-resolved** y **avahi**).
- El **DNS API** (o el **dns_api_mock**) accesible, con **DNS_API_URL** apuntándole (p. ej. **http://host.docker.internal:8080**).
- Las **semillas** cargadas: un registro de cada tipo (**single**, **multi**, **weight**, **geo**) y el registro round-trip de **dns_interceptor/round_trip_seeds.sql**.

En los comandos se asume el puerto de host **15353** (**dig** lo fija con **-p**; el sistema operativo siempre usa el 53, ver la prueba 8).

El script `dns_interceptor/pruebas_interceptor.sh` automatiza las pruebas 3, 5 y 6 (una consulta por tipo de registro, dominio externo y concurrencia):

```
./dns_interceptor/pruebas_interceptor.sh 127.0.0.1 15353
```

1. Recepción e inspección del paquete DNS crudo

Qué prueba: el interceptor recibe el datagrama UDP y sus bytes siguen la estructura DNS (header de 12 bytes, `ID`, flags, `QNAME` en labels).

Con el buffer imprimiéndose en hex (`println!("{:02x?}", &buffer[..amt])`):

```
dig @127.0.0.1 -p 15353 example.com
sudo tcpdump -i any -n udp port 15353 -X # opcional: ver los bytes en el cable
```

Esperado: se imprimen los bytes; se reconocen `ID` (0–1), flags (2–3) y `07 example 03 com 00` en el `QNAME`.

2. Extracción del header y bifurcación estándar / no estándar

Qué prueba: el interceptor extrae `QR` y `Opcode` y solo procesa consultas estándar (`QR=0`, `Opcode=0`).

```
dig @127.0.0.1 -p 15353 example.com # estándar -> se procesa
dig @127.0.0.1 -p 15353 +opcode=STATUS example.com # no estándar -> se descarta
```

Esperado: la segunda genera `No estandar query` en el log y no se resuelve.

3. Resolución por tipo de registro

Una estrategia por cada tipo administrado:

- **single** — devuelve su única IP.
- **multi** — round-robin: la IP rota entre consultas.
- **weight** — distribución ponderada (la IP de mayor peso sale más veces).
- **geo** — IP según el país del cliente. Por el NAT de Docker el origen es el gateway (`172.17.0.1` → país `ZZ`), así que cae en la IP de respaldo; para probar un país real: `curl "http://localhost:8080/api/ip_country?ip=200.1.2.3"`.
- **round-trip** — IP de menor latencia según el checker más cercano; con las semillas (30/90/180 ms desde CR-01) devuelve `1.1.1.1`.

```
dig @127.0.0.1 -p 15353 +short single.example.com
for i in $(seq 1 6); do dig @127.0.0.1 -p 15353 +short multi.example.com; done
```

```
for i in $(seq 1 100); do dig @127.0.0.1 -p 15353 +short weight.example.com; done  
| sort | uniq -c  
dig @127.0.0.1 -p 15353 +short geo.example.com  
dig @127.0.0.1 -p 15353 +short rtt.example.com
```

4. Filtrado de IPs no saludables

Qué prueba: solo se devuelven IPs con **healthy: true**. Marcar una IP como **false** en la BD y consultar varias veces: nunca se retorna. Si todas están no saludables, se registra **No se encontraron IPs saludables...** y no responde.

5. Resolución de dominios externos (no administrados)

Qué prueba: un dominio ausente en la BD se reenvía al DNS remoto vía **POST /api/dns_resolver** (BASE64) y su respuesta se devuelve al cliente.

```
dig @127.0.0.1 -p 15353 google.com
```

Esperado: sección **ANSWER** con la(s) IP(s) reales del dominio.

6. Concurrencia

Qué prueba: el interceptor atiende varias consultas a la vez (un hilo por solicitud), sin serializarlas.

```
for i in $(seq 1 10); do dig @127.0.0.1 -p 15353 +short geo.example.com & done;  
wait
```

Esperado: las 10 responden sin bloquearse entre sí; en el log se ven trazas intercaladas.

7. Robustez ante fallos por solicitud

Qué prueba: un error en una consulta (API caída, respuesta inválida, paquete mal formado) no tumba el servidor. Detener el DNS API, consultar un dominio administrado y volver a levantarlo: se registra **Error consultando /api/exists...** pero el interceptor sigue vivo y responde cuando el API regresa.

8. Uso del interceptor como DNS del sistema (prueba end-to-end)

Descripción: verifica el flujo completo usando el interceptor como resolvedor DNS de toda la máquina y navegando por internet.

Cómo replicar:

1. Exponer el interceptor en el puerto 53 del loopback:

```
sudo docker run --rm -it -p 127.0.0.1:53:53/udp \
  --add-host=host.docker.internal:host-gateway \
  -e DNS_API_URL=http://host.docker.internal:8080 \
  -v "$(pwd)/dns_interceptor:/app" -v cargo-target:/app/target -w /app \
  rust:latest cargo run
```

2. Apuntar `systemd-resolved` a `127.0.0.1` en `/etc/systemd/resolved.conf` (`DNS=127.0.0.1`, `Domains=~.`) y reiniciar: `sudo systemctl restart systemd-resolved`.
3. Navegar (`curl -I https://example.com` o abrir un navegador).

Resultado esperado: las páginas cargan; en el log del interceptor se ve el alto volumen de consultas del sistema. Para revertir, quitar las líneas de `resolved.conf` y reiniciar `systemd-resolved`.

Limitación conocida: el interceptor solo maneja UDP; respuestas que requieran TCP (truncación / bit `TC`) o dominios muy grandes pueden fallar.

DNS API

DNS UI

Las pruebas se hicieron con el DNS API disponible en `http://localhost:8080`.

1. Iniciar la interfaz

Desde la carpeta principal del proyecto:

```
cd dns-ui
docker compose up --build
```

Abrir `http://localhost:3000`. La página debe cargar y mostrar los registros DNS.

2. Probar los registros DNS

1. Crear un registro de cada tipo: `single`, `multi`, `weight`, `round-trip` y `geo`.
2. Editar uno de los registros.
3. Eliminar uno de los registros.

Resultado esperado: los cambios se muestran en la tabla y quedan guardados en el DNS API.

3. Probar los health checks

1. Crear un registro con una prueba TCP.
2. Editar el registro y cambiar la prueba a HTTP.
3. Guardar los cambios.

Resultado esperado: la interfaz guarda los datos de cada prueba sin errores.

4. Probar los rangos IP por país

1. Crear un rango de IP y asignarle un país.
2. Editar el rango.
3. Eliminar el rango.

Resultado esperado: los cambios se muestran en la tabla de rangos IP.

5. Revisar el código

```
cd dns-ui
npm ci
npm run lint
npm run build
```

Resultado esperado: los comandos terminan sin errores.

Health Checker

Video de demostración

«»

Conclusiones y recomendaciones

Conclusiones

1. Es posible lograr que el interceptor opere end-to-end como DNS del sistema. En algunas páginas la latencia del de la plataforma puede hacer que no se logre acceder con la comodidad de un resolver normal, pero sí es posible conseguir conexión a internet con esta implementación.
2. El modelo de concurrencia en Rust (`UdpSocket` compartido con `Arc` y un hilo por solicitud) dio un servidor concurrente y resiliente: un fallo en una consulta se registra sin detener el proceso.
3. La selección `round-trip` (checker más cercano por haversine + menor latencia) se validó con mediciones simuladas, confirmando el uso de los health checkers como proxy de la ubicación del cliente.

Recomendaciones

1. Si queda tiempo durante la implementación sería bueno implementar soporte TCP para respuestas que excedan los 512 bytes de UDP; es la limitación principal actual.
2. Agregar validación de límites (bounds checks) al indexado de bytes en `extraer_host/construir_respuesta` para blindar el parseo ante paquetes mal formados.
3. Tratar de disminuir la latencia de la plataforma a la hora de resolver dominios externos es clave para lograr una utilización cómo en caso de querer utilizar el intercpetor como DNS del sistema.